

Bucket Sort and Applications

Kinza Afzal

November 3, 2025

1 Introduction to Bucket Sort

Bucket Sort is a **distribution sorting algorithm** that divides the input array into multiple buckets and then sorts each bucket individually, often using **Insertion Sort**. Finally, all buckets are concatenated to get the sorted array.

Concept

- Used when input is uniformly distributed over a range (e.g., floating-point numbers between 0 and 1).
- Idea: Distribute elements into buckets, sort each bucket, and combine results.

2 Steps of Bucket Sort

1. Create n empty buckets.
2. Distribute array elements into buckets based on their range.
3. Sort each bucket using Insertion Sort.
4. Concatenate all buckets into one sorted array.

3 C++ Implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Insertion Sort for individual buckets (using swap)
5 void insertionSort(vector<float>& bucket) {
6     for (int i = 1; i < bucket.size(); i++) {
7         int j = i;
8         while (j > 0 && bucket[j] < bucket[j - 1]) {
9             swap(bucket[j], bucket[j - 1]);
10            j--;
11        }
12    }
13 }
```

```

14
15 void bucketSort(float arr[], int n) {
16     vector<vector<float>> buckets(n);
17
18     // Step 1: Distribute elements
19     for (int i = 0; i < n; i++) {
20         int index = n * arr[i];
21         buckets[index].push_back(arr[i]);
22     }
23
24     // Step 2: Sort each bucket
25     for (int i = 0; i < n; i++)
26         insertionSort(buckets[i]);
27
28     // Step 3: Merge all buckets
29     int idx = 0;
30     for (int i = 0; i < n; i++) {
31         for (float val : buckets[i])
32             arr[idx++] = val;
33     }
34 }
35
36 int main() {
37     float arr[] = {0.42, 0.32, 0.23, 0.52, 0.25, 0.47, 0.51};
38     int n = 7;
39     bucketSort(arr, n);
40     cout << "Sorted array: ";
41     for (float x : arr) cout << x << " ";
42     return 0;
43 }

```

4 Dry Run Example

Input: [0.42, 0.32, 0.23, 0.52, 0.25, 0.47, 0.51]

Step 1: Distribute into Buckets (7 buckets total)

Bucket Index	Contents
0	[0.23, 0.25]
1	[0.32]
2	[0.42, 0.47, 0.51, 0.52]
3 – 6	[]

Step 2: Sort each Bucket using Insertion Sort Buckets remain almost sorted due to uniform distribution.

Step 3: Merge buckets → [0.23, 0.25, 0.32, 0.42, 0.47, 0.51, 0.52]

5 Complexity Analysis

Case	Time Complexity
<i>Best</i>	$O(n + k)$
<i>Average</i>	$O(n + k)$
<i>Worst</i>	$O(n^2)$
<i>Space</i>	$O(n + k)$

where k = number of buckets.

6 LeetCode 347: Top K Frequent Elements (Bucket Sort + HashMap)

Concept: Use a HashMap to count frequencies, and use Bucket Sort to group numbers by their frequencies.

C++ Code

```
1 class Solution {
2 public:
3     vector<int> topKFrequent(vector<int>& nums, int k) {
4         unordered_map<int,int> freq;
5         for (int num : nums)
6             freq[num]++;
7
8         int n = nums.size();
9         vector<vector<int>> buckets(n + 1);
10        for (auto& p : freq)
11            buckets[p.second].push_back(p.first);
12
13        vector<int> res;
14        for (int i = n; i >= 0 && res.size() < k; i--) {
15            for (int num : buckets[i]) {
16                res.push_back(num);
17                if (res.size() == k) break;
18            }
19        }
20        return res;
21    }
22};
```

Dry Run

Input: nums = [1,1,1,2,2,3], k = 2

Frequency Map: {1 : 3, 2 : 2, 3 : 1}

	Index (Frequency)	Elements
Buckets:	1	[3]
	2	[2]
	3	[1]

Traverse from high to low:

Result: [1, 2]

Complexity

$O(n)$ time, $O(n)$ space.

7 Alternative Solution: HashMap + Max Heap

Idea: Use a frequency map and a max heap to extract top K frequent elements.

```

1  class Solution {
2  public:
3      vector<int> topKFrequent(vector<int>& nums, int k) {
4          unordered_map<int, int> freq;
5          for (int num : nums)
6              freq[num]++;
7
8          priority_queue<pair<int, int>> maxHeap;
9          for (auto& p : freq)
10             maxHeap.push({p.second, p.first});
11
12         vector<int> res;
13         while (k--) {
14             res.push_back(maxHeap.top().second);
15             maxHeap.pop();
16         }
17         return res;
18     }
19 };

```

Dry Run

nums = [1,1,1,2,2,3], k = 2

Frequency Map: {1 : 3, 2 : 2, 3 : 1}

Heap Contents: [(3, 1), (2, 2), (1, 3)]

Extract top 2 → [1, 2]

Complexity

$O(n \log n)$ time, $O(n)$ space.

8 Comparison of Both Methods

Approach	Time Complexity	Space Complexity
<i>HashMap + BucketSort</i>	$O(n)$	$O(n)$
<i>HashMap + MaxHeap</i>	$O(n \log n)$	$O(n)$

9 LeetCode 451: Sort Characters By Frequency

Bucket Sort + HashMap Approach

```
1 class Solution {
2 public:
3     string frequencySort(string s) {
4         unordered_map<char,int> freq;
5         for(char c : s) freq[c]++;
6
7         int maxFreq = 0;
8         for(auto &p : freq) maxFreq = max(maxFreq,p.second);
9
10        vector<vector<char>> buckets(maxFreq+1);
11        for(auto &p : freq) buckets[p.second].push_back(p.first);
12
13        string res = "";
14        for(int i=maxFreq;i>=1;i--) {
15            for(char c : buckets[i]) res += string(i,c);
16        }
17        return res;
18    }
19};
```

Dry Run (Bucket Sort)

Input: "tree"

Frequency Map: $\{t : 1, r : 1, e : 2\}$

	Frequency	Characters
Buckets:	1	$[t, r]$
	2	$[e]$

Traverse high $\rightarrow$ low $\rightarrow$ Result: "eetr" or "eert"

HashMap + Max Heap Approach

```
1 class Solution {
2 public:
3     string frequencySort(string s) {
4         unordered_map<char,int> freq;
5         for(char c : s) freq[c]++;
6         priority_queue<pair<int,char>> maxHeap;
7         for(auto &p : freq) maxHeap.push({p.second,p.first});
8
9         string res = "";
10        while(!maxHeap.empty()) {
11            auto top = maxHeap.top(); maxHeap.pop();
12            res += string(top.first,top.second);
13        }
14        return res;
15    }
16};
```

Dry Run (Max Heap)

Input: "tree"

Heap: [(2,e),(1,t),(1,r)] → pop → append 'e'*2, 't', 'r' → Result: "eetr"

10 Complexity Comparison

Approach	Time Complexity	Space Complexity
<i>BucketSort + HashMap</i>	$O(n)$	$O(n)$
<i>HashMap + MaxHeap</i>	$O(n \log n)$	$O(n)$